

RAG Chatbot: Source Code & Deployment Manual

Full implementation files (app.py, requirements.txt, README.md) and deployment instructions for GitHub & Streamlit Community Cloud.

1. Complete Project File Structure

```
rag-streamlit-chatbot/  
|  
├─ app.py           # Main Streamlit Web Application  
├─ requirements.txt  # Dependencies list  
├─ README.md        # Documentation  
├─ .gitignore       # Git ignored files  
├─ .streamlit/  
  └─ config.toml    # Streamlit settings
```

2. Complete Runnable `app.py`

```

import os, io, numpy as np, streamlit as st, faiss
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer
from groq import Groq

DEFAULT_GROQ_MODEL = "llama-3.3-70b-versatile"
EMBEDDING_MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"

st.set_page_config(page_title="RAG Document Chatbot", page_icon="📄", layout="wide")

def get_groq_api_key():
    try:
        if "GROQ_API_KEY" in st.secrets and st.secrets["GROQ_API_KEY"]:
            return st.secrets["GROQ_API_KEY"]
    except Exception:
        pass
    return os.getenv("GROQ_API_KEY", "")

@st.cache_resource(show_spinner="Loading embedding model...")
def load_embedding_model():
    return SentenceTransformer(EMBEDDING_MODEL_NAME)

def extract_text(uploaded_files):
    docs, scanned = [], []
    for f in uploaded_files:
        content = f.read()
        if f.name.endswith(".pdf"):
            reader = PdfReader(io.BytesIO(content))
            has_txt = False
            for p_num, page in enumerate(reader.pages, start=1):
                txt = (page.extract_text() or "").strip()
                if txt:
                    has_txt = True
                    docs.append({"filename": f.name, "page": p_num, "text": txt})
            if not has_txt: scanned.append(f.name)
        elif f.name.endswith(".txt"):
            txt = content.decode("utf-8", errors="ignore").strip()
            if txt: docs.append({"filename": f.name, "page": 1, "text": txt})
    return docs, scanned

def create_chunks(docs, size, overlap):
    chunks = []
    for doc in docs:
        text, filename, page = doc["text"], doc["filename"], doc["page"]
        start, idx = 0, 1
        while start < len(text):
            chunks.append({
                "chunk_id": f"{filename}_p{page}_c{idx}",
                "filename": filename, "page": page, "chunk_num": idx,
                "text": text[start:start+size]
            })
            idx += 1
            start += (size - overlap)
    return chunks

def build_index(chunks, embedder):
    if not chunks: return None, []
    texts = [c["text"] for c in chunks]
    embeddings = embedder.encode(texts, convert_to_numpy=True)
    faiss.normalize_L2(embeddings)
    index = faiss.IndexFlatIP(embeddings.shape[1])
    index.add(embeddings)
    return index, chunks

def retrieve(query, index, chunks, embedder, top_k):

```

```

    if index is None or not chunks: return []
    q_vec = embedder.encode([query], convert_to_numpy=True)
    faiss.normalize_L2(q_vec)
    distances, indices = index.search(q_vec, top_k)
    return [chunks[i] for i in indices[0] if i != -1 and i < len(chunks)]

def generate_ans(query, chunks, history, model, api_key):
    client = Groq(api_key=api_key)
    ctx = "".join([f"Source: {c['filename']} (p.{c['page']})"
{c['text']}
" for c in chunks])
    sys_prompt = "Answer STRICTLY from context. If not found, output: 'I could not find this
information in the uploaded documents.'"
    res = client.chat.completions.create(
        model=model,
        messages=[{"role": "system", "content": sys_prompt}, {"role": "user", "content": f"Context:
{ctx}
Question: {query}"}],
        temperature=0.1
    )
    return res.choices[0].message.content

# Session State & UI Logic
if "messages" not in st.session_state: st.session_state.messages = []
if "faiss_index" not in st.session_state: st.session_state.faiss_index = None
if "chunks_db" not in st.session_state: st.session_state.chunks_db = []
if "kb_ready" not in st.session_state: st.session_state.kb_ready = False

st.title("📄 RAG Document Chatbot")
api_key = get_groq_api_key()

with st.sidebar:
    st.header("Upload & Settings")
    uploaded = st.file_uploader("Upload PDF/TXT", type=["pdf", "txt"], accept_multiple_files=True)
    chunk_size = st.slider("Chunk Size", 500, 1500, 900)
    chunk_overlap = st.slider("Overlap", 50, 300, 150)
    top_k = st.slider("Top K", 1, 10, 4)

    if st.button("Process Documents", type="primary"):
        if uploaded and api_key:
            embedder = load_embedding_model()
            docs, scanned = extract_text(uploaded)
            chunks = create_chunks(docs, chunk_size, chunk_overlap)
            index, db = build_index(chunks, embedder)
            st.session_state.faiss_index = index
            st.session_state.chunks_db = db
            st.session_state.kb_ready = True
            st.success("Knowledge Base Ready!")

# Render Chat
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]): st.markdown(msg["content"])

if prompt := st.chat_input("Ask a question..."):
    if st.session_state.kb_ready and api_key:
        st.session_state.messages.append({"role": "user", "content": prompt})
        with st.chat_message("user"): st.markdown(prompt)

        embedder = load_embedding_model()
        retrieved = retrieve(prompt, st.session_state.faiss_index, st.session_state.chunks_db,
embedder, top_k)
        ans = generate_ans(prompt, retrieved, st.session_state.messages, DEFAULT_GROQ_MODEL, api_key)

        with st.chat_message("assistant"):

```

```

st.markdown(ans)
with st.expander("Sources"):
    for r in retrieved: st.caption(f"{r['filename']} (p.{r['page']})")
st.session_state.messages.append({"role": "assistant", "content": ans})

```

3. Configuration Files

`requirements.txt`

```

streamlit>=1.35.0
groq>=0.9.0
sentence-transformers>=3.0.0
faiss-cpu>=1.8.0
pypdf>=4.2.0
numpy>=1.26.0

```

`.gitignore`

```

__pycache__/
*.py[cod]
.env
.streamlit/secrets.toml
venv/

```

4. Step-by-Step GitHub & Streamlit Cloud Deployment

1. **Create GitHub Repository:** Go to github.com -> New Repository -> Name: `rag-streamlit-chatbot` -> Public.
2. **Upload Files:** Add `app.py`, `requirements.txt`, `README.md`, and `.gitignore`.
3. **Deploy on Streamlit Cloud:**
 - Go to share.streamlit.io and sign in with GitHub.
 - Click **New App** -> Select repository and `app.py`.
 - Open **Advanced Settings** -> **Secrets** and paste:

```
GROQ_API_KEY = "gsk_your_actual_key_here"
```

- Click **Deploy!**